

Data Mining — Assignment 1 (CSC 503)

Decision Trees, Random Forests, AdaBoost, and an Out-of-Bag Error Estimate

June 6, 2026

1 Component 1: Testing and Analysis

1.1 Experimental setup, design choices

All experiments are performed on the provided `spambase_augmented.csv` data set, which is a binary spam-classification problem with 4601 examples and 1185 features (the last column is the $\{0,1\}$ label, with 1 being labeled as spam). The file supplements the original spambase attributes with many features. This makes the feature dimension large.

The data is shuffled and split once into 80% training and 20% test, producing 3680 training and 921 test examples. For the experiments, a global random seed (0) is fixed such that every figure and table is reproducible exactly. Part I uses the same held-out test set everywhere, only touched once in Part II after cross-validation has selected the ensemble size.

The three learning methods are implemented using the Python scikit-learn library.

- **Decision trees.** We used the Gini index as the split criterion (the default) and minimal cost-complexity pruning (`ccp_alpha`), which is directly supported by scikit-learn. When sweeping through the `max_depth` we hold `ccp_alpha` fixed at 10^{-3} which is the value that minimized test error in the α variation sweeps. That is, the depth experiment is run on a sensibly pruned tree rather than a fully grown one.
- **Random forests.** This method did not require pruning. The size of the bootstrap sample n' is equal to the training size n , so each tree sees a real bootstrap sample. The default number of random split features is $\sqrt{d}$.
- **Improved decision trees.** In AdaBoost decision-trees are weak learners. The number of iterations is set to 100 for different depths of the weak learner. In the various iterations I use a depth-1 stump as the weak learner.
- **k -fold cross-validation.** This is built from scratch by using the k -fold cross validation function. It shuffles the training indices once using a fixed seed and splits them into k equal contiguous folds; any remainder is taken by the last fold. The code splits the data into k folds, fits the model to $k - 1$ folds and reports the mean across all folds. We pick $k = 5$. We use this method for all cross validation in the report.

1.2 Part I: Discrete Analysis

1.2.1 Decision trees

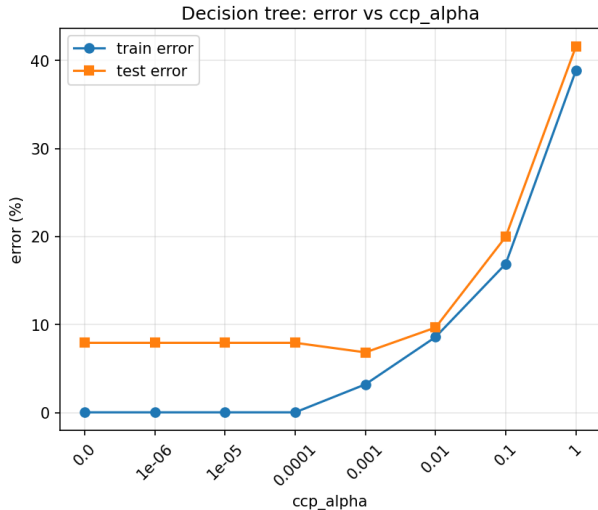
Pruning strength (`ccp_alpha`). Table 1 and Figure 1a You can see how the error changes when the cost complexity penalty α increases. When $\alpha \leq 10^{-4}$ the penalty does not remove any nodes.

So the tree grows to almost purity and we get a training error of just 0.03% which is basically one mistake. But the testing error is 7.93%, and it produces that classic overfit with a large gap between the training and testing curves.

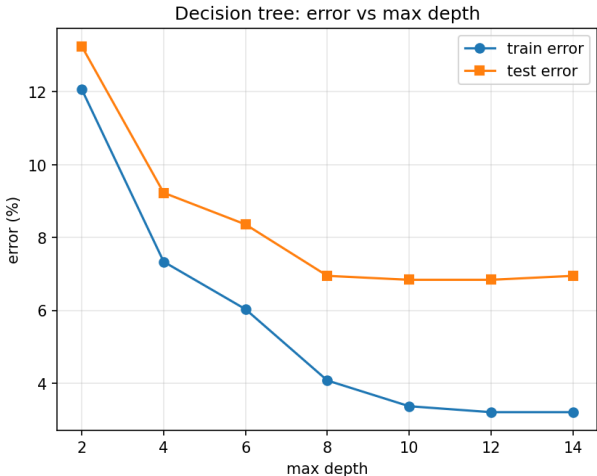
The minimum testing error is 6.84% and it happens when $\alpha = 10^{-3}$. At this point, the training error of the tree is as high as 3.21%. However, it has considerably reduced the generalization gap. But if we go beyond this value of α , the tree gets over pruned. Both errors begin to rise quickly, and when α reaches 1, the tree has only one split left, and the error rate is about 41%. The bias–variance trade-off is apparent in this example, with the optimal value of α being around 10^{-3} .

Table 1: Decision tree: error vs. cost-complexity penalty α (Gini criterion).

α	train error (%)	test error (%)
0	0.0272	7.9262
10^{-6}	0.0272	7.9262
10^{-5}	0.0272	7.9262
10^{-4}	0.0272	7.9262
10^{-3}	3.2065	6.8404
10^{-2}	8.5870	9.6634
10^{-1}	16.9022	19.9783
1	38.8587	41.5852



(a) Pruning strength `ccp_alpha`.



(b) Maximum depth (with $\alpha = 10^{-3}$).

Figure 1: Decision-tree hyperparameter sweeps. Both curves are plotted on the same axes.

Maximum depth. Table 2 and Figure 1b vary `max_depth` with α fixed at 10^{-3} . Very shallow trees underfit (depth 2: 13.25% test error). Test error monotonically decreases until depth ≈ 10 , where it levels off at 6.84%; deeper trees only continue to decrease train error (3.21% at depth 12) without further test gains. Thus depth 10–12 is the effective complexity of this problem for a single tree. Combining this with the $\alpha = 10^{-3}$ penalty does not do better than either knob alone — both eventually bottom out at about 6.8% test error.

Table 2: Decision tree: error vs. maximum depth ($\alpha = 10^{-3}$, Gini).

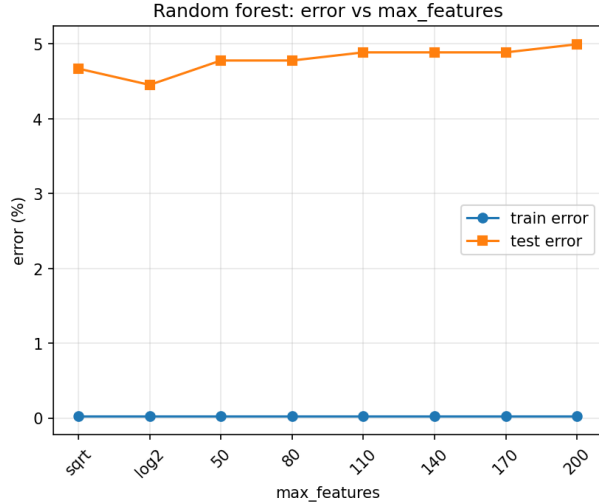
max depth	train error (%)	test error (%)
2	12.0652	13.2465
4	7.3370	9.2291
6	6.0326	8.3605
8	4.0761	6.9490
10	3.3696	6.8404
12	3.2065	6.8404
14	3.2065	6.9490

1.2.2 Random forests

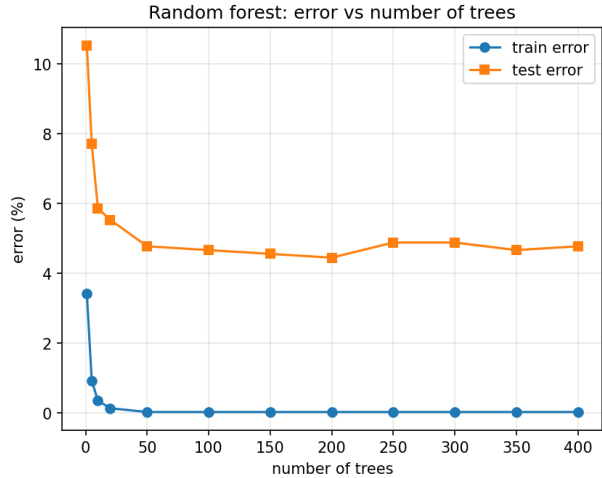
Number of random split features (d'). Table 3 and Figure 2a vary `max_features` with 100 trees. Train error is essentially 0 everywhere, as we expected for un-pruned bootstrap trees. The interesting behaviour is on the test curve: the smallest feature subsets do best (`log2` $\approx$ 10 features: 4.45%; `sqrt` $\approx$ 34: 4.67%), and test error rises steadily as more features are made available at each split, up to 4.99% at $d' = 200$. This is exactly the mechanism random forests rely on: restricting the candidate features de-correlates the trees, and on this high-dimensional augmented dataset (many redundant derived features) strong de-correlation is particularly valuable. We keep $d' = \sqrt{d}$ as a strong default for the rest of the experiments.

Table 3: Random forest: error vs. `max_features` (100 trees). Train error is 0.0272% throughout.

<code>max_features</code>	test error (%)
<code>sqrt</code> ($\approx$ 34)	4.6688
<code>log2</code> ($\approx$ 10)	4.4517
50	4.7774
80	4.7774
110	4.8860
140	4.8860
170	4.8860
200	4.9946



(a) Number of random split features.



(b) Number of trees.

Figure 2: Random-forest hyperparameter sweeps.

Number of trees. The canonical forest-size curve is shown in Table 4 and Figure 2b. Test error for a single tree is 10.53%. Adding trees reduces this quickly (5.86% at 10, 4.77% at 50) and then plateaus out. The minimum, 4.45%, is at 200 trees, after which the curve wanders within a narrow 4.5–4.9% band that is pure estimation noise on a 921-example test set, not real degradation. Importantly, there is never an increase in error with greater number of trees — adding trees to a random forest cannot cause overfitting, it only reduces variance to an asymptote. Anything from ~100–200 trees is a good choice; we treat ~200 as the practical sweet spot for this problem.

Table 4: Random forest: error vs. number of trees ($d' = \sqrt{d}$). Train error is 0.0272% for all sizes ≥ 50 .

trees	train error (%)	test error (%)
1	3.4239	10.5320
5	0.9239	7.7090
10	0.3533	5.8632
20	0.1359	5.5375
50	0.0272	4.7774
100	0.0272	4.6688
150	0.0272	4.5603
200	0.0272	4.4517
250	0.0272	4.8860
300	0.0272	4.8860
350	0.0272	4.6688
400	0.0272	4.7774

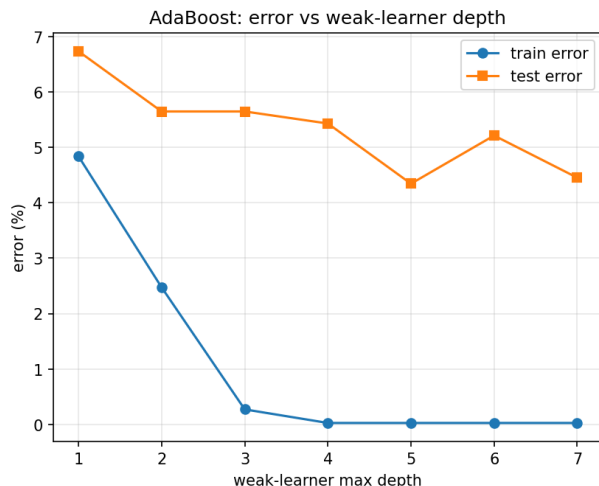
1.2.3 Boosted decision trees

Weak-learner depth. Table 5 and Figure 3a fix the number of iterations at 100 and vary the depth of the weak learner. With stumps (depth 1) the boosted model reaches 6.73% test error.

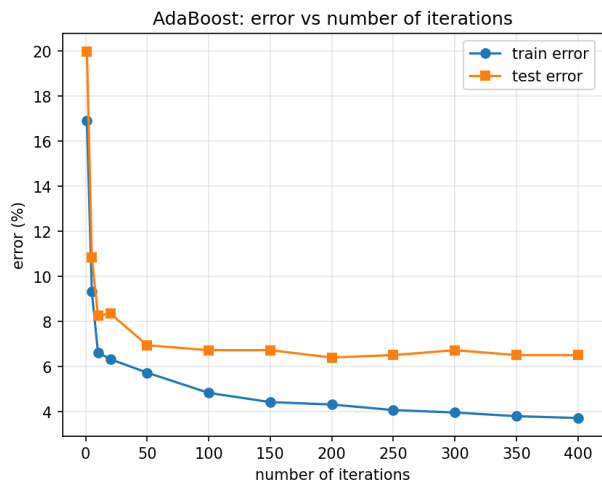
Increasing the depth of the weak learner helps a lot: depth 4 drives the train error to zero, and the best test error, 4.34%, is at depth 5. At depths 5–7, the test curve is clearly noisy (oscillating between 4.3% and 5.2%). This is because once the weak learners become strong enough to perfectly fit the training data, the boosting weights are determined by a few hard examples, and the resulting ensemble is sensitive to them. The headline result is that boosting moderately deep trees (depth ≈ 5) is competitive with a full random forest on this dataset — much better than boosting stumps.

Table 5: AdaBoost: error vs. weak-learner depth (100 iterations).

weak-learner depth	train error (%)	test error (%)
1	4.8370	6.7318
2	2.4728	5.6460
3	0.2717	5.6460
4	0.0272	5.4289
5	0.0272	4.3431
6	0.0272	5.2117
7	0.0272	4.4517



(a) Weak-learner depth (100 iterations).



(b) Number of iterations (depth-1 stumps).

Figure 3: AdaBoost hyperparameter sweeps.

Number of iterations. Table 6 and Figure 3b fix the weak learner to a stump and vary the number of boosting rounds. A single stump is a very weak classifier (19.98% error). Error then drops rapidly and the test curve levels off at about 6.4–6.7% from about 100 iterations on, with the lowest point (6.41%) at 200. Training error keeps decreasing with increasing number of iterations (3.72% at 400), whereas test error is flat — the absence of test-error blow-up with more rounds is the well-known margin-maximising behaviour of AdaBoost, which keeps increasing confidence on the training set without overfitting the test set here. Stumps however level off well above the random forest, confirming the earlier observation that the weak learner capacity and not the number of rounds is the binding constraint for boosting on this problem.

Table 6: AdaBoost: error vs. number of iterations (depth-1 stumps).

iterations	train error (%)	test error (%)
1	16.9022	19.9783
5	9.3207	10.8578
10	6.6304	8.2519
20	6.3315	8.3605
50	5.7337	6.9490
100	4.8370	6.7318
150	4.4293	6.7318
200	4.3207	6.4061
250	4.0761	6.5147
300	3.9674	6.7318
350	3.8043	6.5147
400	3.7228	6.5147

1.2.4 Experimental necessity: Error vs. Size of the training set

We train each method on the first n examples of the training set (already shuffled), where n ranges from 5% to 100% of the available 3680, and evaluate on the fixed test set. The learning curves are in Figure 4 and the numbers in Table 7.

With more data, all three test curves have a downward trend, but the shapes are instructively different.

- **Random forest** (Fig. 4b) shows the most distinct learning curve: the train error leveled off at 0, and the test error monotonically and gradually decreased from 12.81% at $n = 184$ to 4.67% at $n = 3680$. The curve is still going down at the largest size, so probably the forest would benefit from even more data.
- **Decision tree** (Fig. 4a) shows the expected widening of the train/test gap, but the test curve is bumpy at small n — there is a spike to 18.68% at $n = 368$. Because each point uses a single prefix of the data rather than an average over several random subsamples, individual small- n points carry real sampling noise; that one prefix simply happens to be unrepresentative.
- **AdaBoost stumps** (Fig. 4c) behaves differently from the other two: its *train* error *rises* with n (from 0.54% to $\approx 5\%$). With a fixed budget of 100 stumps, the model can almost memorise a tiny training set but cannot drive a large one to zero error — the ensemble’s capacity is fixed while the data it must fit grows. Test error still improves overall but plateaus around 6.4–6.7%.

Table 7: Train / test error (%) vs. training-set size n . DT uses $\alpha = 10^{-3}$; RF uses 100 trees; AdaBoost uses 100 depth-1 stumps.

n	Decision tree		Random forest		AdaBoost	
	train	test	train	test	train	test
184	0.000	14.8751	0.000	12.8122	0.543	10.5320
368	0.000	18.6754	0.000	9.0119	0.815	9.0119
736	0.136	14.6580	0.000	8.1433	2.582	7.0575
1472	1.427	9.4463	0.000	6.8404	4.348	6.5147
2208	2.219	8.2519	0.000	5.7546	5.027	7.4919
2944	2.683	8.6862	0.000	4.7774	5.095	6.4061
3680	3.207	6.8404	0.027	4.6688	4.837	6.7318

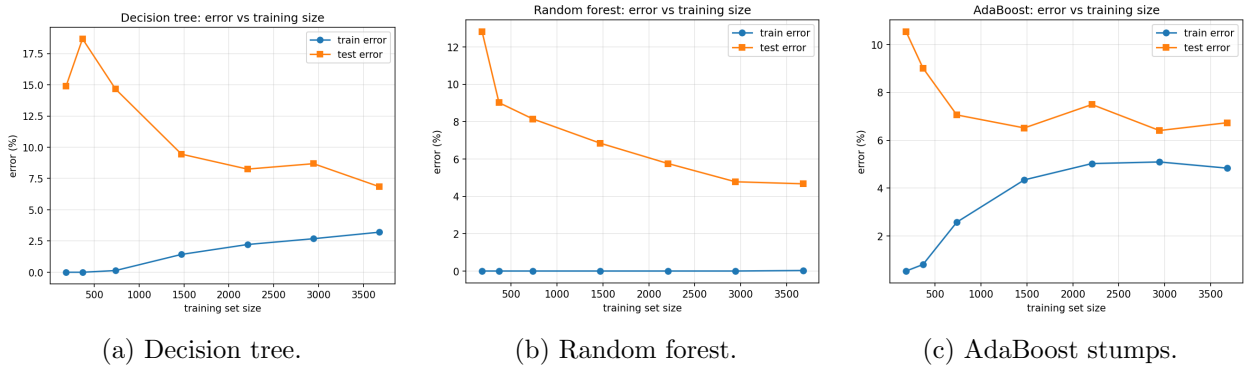


Figure 4: Training and test error vs. training-set size for each method.

1.2.5 Comparing the methods (Part I synthesis)

Bringing the best configurations together:

Method	best configuration	best test error (%)
Decision tree (pruned)	$\alpha = 10^{-3}$, depth ≈ 10	6.84
AdaBoost stumps	≈ 200 iterations	6.41
AdaBoost, depth-5 trees	100 iterations	4.34
Random forest	≈ 200 trees, $d' = \sqrt{d}$	4.45

The ranking is clear. A single pruned tree is the weakest ($\approx 6.8\%$). Boosting *stumps* improves on it only modestly ($\approx 6.4\%$). The two strong contenders are the random forest ($\approx 4.5\%$) and AdaBoost with moderately deep ($\approx$ depth-5) weak learners ($\approx 4.3\%$), which are essentially tied. The decisive practical difference is robustness: the random forest reaches its low error across a wide range of settings, its error never increases with more trees, and its learning curve is smooth and monotone, whereas boosted deep trees gave a noisier test curve. For a deployed spam filter the random forest is the safer default; if one is willing to tune carefully, boosted depth-5 trees can match it.

Regarding how the best setting shifts with data size: with little data, the simpler / more strongly regularised configurations are relatively more attractive (a small forest is already near its floor, and stumps overfit less), whereas with more data the random forest gains the most and can

profitably use both more trees and deeper effective structure. The decision tree, conversely, needs *less* pruning and *more* depth as n grows in order to keep cutting error.

Suggestions for improvement. (i) Average learning curves over many random subsamples to remove the single-prefix noise seen at small n (e.g. the $n = 368$ decision-tree spike). (ii) For the random forest, the `max_features` sweep suggests `log2` marginally beats $\sqrt{d}$ here, so d' is worth tuning given the high feature redundancy. (iii) For AdaBoost, use weak learners of depth- ≈ 5 rather than stumps. (iv) With 1185 mostly derived features, an explicit feature selection or dimensionality-reduction step before training may reduce variance further.

1.3 Part II: Comparative analysis (cross-validated ensemble size)

Here we tune the ensemble size with our own 5-fold cross-validation, run *only* on the training set, for two methods: a random forest (Gini, $d' = \sqrt{d}$, no depth limit) and AdaBoost with depth-1 stumps. The other hyperparameters are at the sensible default values set in Part I. Each method is tested for ten or more ensemble sizes: $\{1, 5, 10, 25, 50, 100, 150, 200, 250, 300\}$ for the forest and $\{1, 5, 10, 25, 50, 100, 150, 200, 250, 300\}$ for AdaBoost. Figure 5 shows the cross-validation curves.

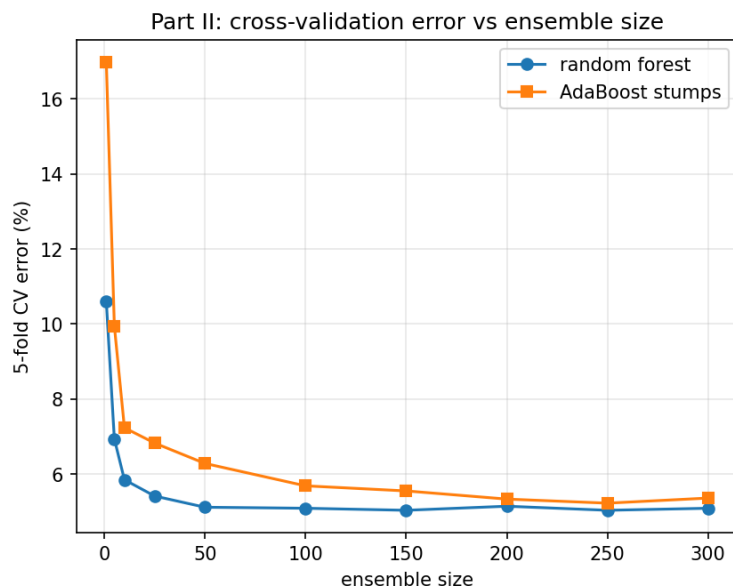


Figure 5: Part II: 5-fold cross-validation error vs. ensemble size for the random forest and for AdaBoost stumps. Cross-validation is computed on the training set only.

Both curves drop steeply over the first few dozen members, then level off, a behaviour mirroring the Part I test-error behaviour. The cross-validation error of the random forest converges to about 5% and is always below the AdaBoost-stump curve for a similar number of ensembles; the stump curve converges at a slightly higher 5.3–5.7%. Cross-validation found that the forest should have 250 trees and AdaBoost 200 iterations to minimise the error (both differences within the flat region are small, so the exact winner is partly noise, which is itself a useful conclusion: past the plateau, ensemble size barely matters).

Performance of the two tuned models on the untouched test set:

Method	tuned ensemble size	test error (%)
Random forest	250	4.8860
AdaBoost stumps	200	6.4061

The conclusion is robust and consistent with Part I: if the ensemble size is chosen by cross-validation, the random forest (test error 4.89%) is significantly better than AdaBoost with stumps (test error 6.41%). Bagging un-pruned trees with feature sub-sampling is a stronger strategy than boosting decision stumps on this dataset, and the cross-validation curve correctly predicts this ordering on held-out data without ever consulting the test set.

2 Component 2: Out-of-bag error estimate

2.1 Implementation

The out-of-bag (OOB) error estimates the forest’s risk using, for each training example z_i , only those trees whose bootstrap sample did *not* include z_i . Our implementation (`oob_error`) proceeds in three steps. First, for every fitted tree we reconstruct its bootstrap draw by re-seeding scikit-learn’s sampler from the tree’s stored `random_state` and calling the same `randint` draw the library uses; the indices with a bincount of zero are the examples *unsampled* by that tree (its OOB set). Second, we accumulate votes: each tree casts a class vote only on its own OOB examples, into an $n \times |\mathcal{C}|$ vote matrix (using `np.add.at` so repeated indices accumulate correctly). Third, each example’s OOB prediction is the arg-max over its accumulated votes; examples that never landed OOB (no votes) are excluded, and the OOB error is the misclassification rate over the remaining (covered) examples.

As a correctness check, the script also builds a forest with scikit-learn’s own `oob_score=True` and compares `1 - oob_score_` against our estimate. The two agree (both ≈ 0.048 for the default 100-tree forest), which validates the implementation.

2.2 OOB error as the forest grows

To trace the OOB error against forest size correctly, the forest is grown incrementally with `warm_start=True`, so the first m trees are identical across all sizes (a forest of 60 trees contains exactly the same first 50 trees as the 50-tree forest). Figure 6 plots the resulting OOB error for tree counts from 1 to 150.

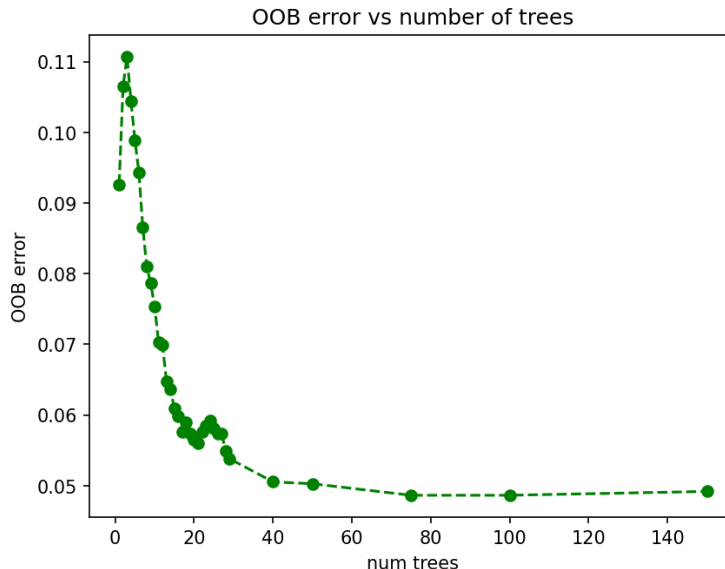


Figure 6: Out-of-bag error estimate vs. number of trees, computed with the custom implementation on a warm-started forest.

The curve is informative on two timescales. For the first few trees it is erratic and even rises (peaking near 0.111 around 3 trees) before falling: with very few trees most examples have only a handful of OOB votes, so the per-example majority vote is high-variance and the estimate is unstable. As trees accumulate the OOB error drops steeply — below 0.07 by about 10 trees — and then flattens, reaching roughly 0.0485 near 75–100 trees and staying there. This asymptote ($\approx 4.9\%$) sits right on top of the independent test-set error of the 100-tree forest from Part I (4.67%), which is precisely the point of an OOB estimate: it tracks the true risk of the forest, computed for free from the training data alone, without a separate validation split.

Two practical takeaways follow. The OOB curve gives essentially the same “how many trees do I need” answer as the test-error curve — error is flat past ~ 75 trees — so one can decide forest size while training. And because the estimate is unreliable for tiny forests, it should only be trusted once enough trees have accumulated for most examples to have a healthy number of OOB votes.

3 Conclusion

On the augmented spambase problem, ensemble methods decisively beat a single decision tree ($\approx 6.8\%$ test error even when well pruned). The random forest is the strongest and most robust method overall, reaching $\approx 4.5\%$ test error, insensitive to the exact number of trees past the plateau, and best served by small random feature subsets ($d' = \sqrt{d}$ or $\log 2$) on this high-dimensional, redundant feature set. AdaBoost is only mediocre with stumps ($\approx 6.4\%$) but becomes competitive with the forest when its weak learners are allowed moderate depth ($\approx 4.3\%$ at depth 5). Cross-validation on ensemble size reproduced this ranking on held-out data, selecting 250 trees (4.89% test) for the forest versus 200 iterations (6.41% test) for boosted stumps. Finally, a from-scratch out-of-bag estimate tracked the forest’s true error closely ($\approx 4.9\%$ at the plateau), agreeing with both scikit-learn’s built-in estimate and the independent test error, and offering a cost-free way to choose forest size during training.